

Lexical & Semantic Retrieval on EB-NeRD and MIND

CS4.406 Information Retrieval & Extraction — Assignment 1 design note · Danish (EB-NeRD, 20,738 articles) and English (MIND, 65,238 articles) news recommendation · all numbers regenerated by `make reproduce` · code: github.com/codewithsajid/ire-a1-news-retrieval

1 · What I built

One pipeline, two datasets, one schema. `python -m newsrec.build --config configs/<name>.yaml` takes raw zips to a parquet feature store; `make reproduce` runs the whole chain — build, leakage tests, BM25 and its (k_1, b) ablation, the ANN index and its parameter sweep, the evaluation harness, the leakage ablation, the leaderboard submissions and every figure in this note.

Canonical schema. `articles` (title, abstract, body, category, entities, published/first-seen time, embeddings), `impressions` (candidates + clicked, both dense u32), `history`, `clickstream`. Columns a dataset genuinely lacks stay null rather than being faked: MIND has no publication date (we derive `first_seen_time` from its earliest impression) and no history timestamps; EB-NeRD has no Wikidata entities.

Temporal split. The last editorial day of each official train week becomes `val`; the official validation split becomes a labelled held-out test; the official test set stays the unlabelled submit split. Strictly increasing, and the official splits keep their published meaning. EB-NeRD's day runs 07:00–07:00, so one editorial day spans two calendar dates — and no session straddles the cut, because that boundary is the day break.

Feature store. Per-split derived features (user recency/activity and category profile; article smoothed CTR and half-life-decayed clicks) each carry a `computed_through` stamp that never reaches into the split they describe. Exposure counts exploit the fact that splits do not overlap: per-split counts are prefix-summed rather than re-exploding 200M candidate rows per split.

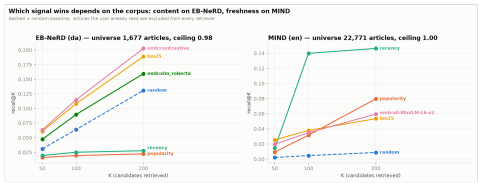
Retrieval. BM25 over a polars postings table (`doc`, `term`, `tf`) → `scipy CSR`; stemming runs once per *distinct token*, not per occurrence. A user's query is their whole click history as a sparse row (§5b), so all 73,152 MIND queries are one matrix product $H \cdot TF$. Semantic side: FAISS over L2-normalised article vectors, user = mean-pooled over that same history — deliberately the same input BM25 gets, so the comparison is about representation and nothing else.

2 · Choices, and what I did not do

Decision	Alternative	Why this one
polars, lazy + streaming	pandas (the starter notebooks)	EB-NeRD large is 13.5M test impressions / 206M candidate slots; the pandas path in the starter notebook does not survive it on this box
Official MRR (mean 1/rank over every click)	Reciprocal rank of the first click	Tested against the graders' own <code>ebrec</code> code. The first-click variant matches only on single-click impressions, so it overstated MIND (28.8% multi-click) by 15% and EB-NeRD (0.51%) by 0.2%; both are reported
Candidates restricted to a 7-day live window, anchored at split <i>start</i>	Whole catalogue; or window anchored at split end	EB-NeRD carries articles from 1993. Anchoring at the end excludes everything already popular when the week began — that bug scored popularity at exactly 0.0000
Drop already-read articles from the retrieved list	Keep them	BM25's top hit was frequently an article from the user's own history; excluding seen items lifts EB-NeRD BM25 recall@50 by +38%
title + abstract as the indexed text	+ body (EB-NeRD only)	Chosen for comparability (MIND ships no body), and later measured: adding the body costs -9% recall for 11× the postings — the comparability argument and the quality argument agree
Exact IndexFlatIP at the operating point	HNSW everywhere	Measured, not assumed — see §4

3 · Retrieval: lexical vs semantic

recall@K against the live candidate universe, seen articles dropped, test split. Ceiling is the share of clicks reachable inside the universe at all: 0.9764 on EB-NeRD (universe 1,677), 1.0000 on MIND (universe 22,771).



Recall@K per method: content wins on EB-NeRD, freshness wins on MIND.

method	EB-NeRD/small			MIND/small		
	@50	@100	@200	@50	@100	@200
random	0.0309	0.0639	0.1308	0.0022	0.0046	0.0087
popularity (prior)	0.0163	0.0194	0.0224	0.0093	0.0317	0.0797
recency	0.0196	0.0251	0.0278	0.0147	0.1399	0.1463
BM25	0.0610	0.1082	0.1888	0.0254	0.0379	0.0534
embeddings	0.0635	0.1148	0.2027	0.0195	0.0349	0.0595

The two datasets disagree, and that is the finding. On EB-NeRD, popularity and recency both fall below random. Only 2 of the 200 most-clicked test-week articles are in the top 200 by prior popularity — they are a median 80 hours younger than the split boundary, so they did not exist when the feature window closed. On MIND the same overlap is

24/200 at a median age of +7h, and recency is the strongest retriever there. A freshness prior is not a dataset-independent trick; it is a bet on how fast the catalogue turns over.

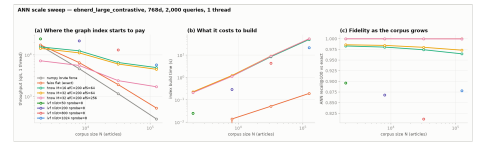
Length normalisation is the only BM25 knob that earns its keep — and only on one of the two datasets. On MIND, taking b from 0 to 1 moves recall@100 from 0.0368 to 0.0626 (+70%). On EB-NeRD the whole b sweep is worth 2.3%, and the optimum is not even stable across corpus size ($b=1.0$ on small, $b=0.3$ on large). The explanation is in the corpora, not the algorithm: EB-NeRD's indexed text is title + subtitle, averaging 16.0 tokens with little variance, so there is barely any length to normalise; MIND's title + abstract averages 30.7 with 5.2% of articles missing the abstract entirely. k_1 is worth 11.4% on MIND and 0.7% on EB-NeRD, because news articles almost never repeat a term and the saturation curve it controls has nothing to bite on. BM25L ($\delta=0.5$) was worse than plain BM25 everywhere. **Tuning BM25 on the demo bundle and shipping it to the leaderboard bundle would have been a mistake here** — which is the argument for tuning on the variant you submit.

Which retriever wins is a fact about the candidate pool, not about the language. Embeddings win on EB-NeRD (0.1065 vs 0.0860) and BM25 wins on MIND (0.0385 vs 0.0357) — but EB-NeRD's live window holds 1,677 articles and MIND's 22,771. Open EB-NeRD's window to its whole catalogue (20,738) and BM25 wins there too (0.0171 vs 0.0120); shrink MIND's to a day (5,793) and embeddings win (0.0772 vs 0.0577). At matched pool sizes the two datasets agree. Absolute recall@K is mostly a statement about that window: it moves $\sim 9\times$ across the sweep while lift over random barely moves.

Lexical and semantic retrieve nearly disjoint sets. Mean top-10 overlap is 0.88/10. BM25 returns near-duplicates of what the user already read; embeddings return related-but-new articles. That is why the rank-fusion hybrid is worth having, and why the right architecture is both, not the better one.

4 • The ANN ablation: does approximation cost anything?

Two questions, different answers. At the **operating point** — a 7-day universe of 1,677 candidates — approximation is not worth it: exact IndexFlatIP runs at 20,463 q/s single-threaded and HNSW at $\text{efSearch}=64$ is *slower* (14,981 q/s), because a graph walk over 1.7K vectors costs more than the matrix multiply it replaces. Task recall is far more forgiving than index fidelity — HNSW down to 63% index recall stays within 4% of the exact task metric, since the ranking only has to be right about the few articles actually clicked — until it collapses: IVF at $nprobe=1$ keeps 2% of the exact ranking and gives up 98% of the task metric with it. The **scale sweep** over the full 125,000-article corpus puts the crossover at $N\approx 8,000$: at $N=125,000$ HNSW runs $9.4\times$ faster than exact FAISS and $17.2\times$ faster than a numpy matmul for 96.5% fidelity and a 50s build. An 8-bit scalar quantiser buys memory, not recall: $4\times$ smaller at 0.998 fidelity, $3.7\times$ slower to scan.



Scale sweep: throughput crossover near $N\approx 8K$, HNSW build cost vs brute force, fidelity given up for that speed.

The crossover at $N\approx 8,000$: at $N=125,000$ HNSW runs $9.4\times$ faster than exact FAISS and $17.2\times$ faster than a numpy matmul for 96.5% fidelity and a 50s build. An 8-bit scalar quantiser buys memory, not recall: $4\times$ smaller at 0.998 fidelity, $3.7\times$ slower to scan.

5 • Ranking: the metric the leaderboards actually score

AUC / MRR / nDCG within each impression's real candidate list, test split. EB-NeRD: 244,647 impressions, 2,928,942 candidate rows. MIND: 73,152 / 2,740,998. 95% CIs from 500 bootstrap resamples over impressions. random is the calibration check — a correct AUC must put it at 0.5000.

dataset	ranker	AUC	MRR	nDCG@5	nDCG@10 (95% CI)	ILD@10	nov@10	cov@10	head/tail
EB-NeRD	random	0.4993	0.3125	0.3447	0.4297 (0.428–0.431)	0.772	15.52	0.926	0.360/0.432
	pop_prior	0.4437	0.2665	0.2918	0.3883 (0.387–0.389)	0.762	15.32	0.898	0.700/0.378
	ctr_prior	0.4245	0.2569	0.2771	0.3782 (0.377–0.379)	0.755	15.34	0.922	0.648/0.370
	recency	0.5015	0.3080	0.3368	0.4219 (0.421–0.423)	0.778	15.64	0.837	0.298/0.426
	bm25	0.5377	0.3479	0.3831	0.4617 (0.460–0.463)	0.762	15.49	0.908	0.430/0.463
	emb	0.5453	0.3522	0.3870	0.4660 (0.465–0.467)	0.673	15.53	0.901	0.376/0.469
	hybrid_rrf	0.5518	0.3541	0.3918	0.4688 (0.467–0.470)	0.708	15.51	0.907	0.405/0.471
	pop_oracle*	0.6540	0.4177	0.4721	0.5326 (0.531–0.534)	0.774	15.61	0.840	0.433/0.536
MIND	random	0.5008	0.2189	0.2240	0.2868 (0.285–0.289)	0.945	15.45	0.663	0.277/0.292
	pop_prior	0.5331	0.2375	0.2466	0.3107 (0.308–0.313)	0.921	13.51	0.356	0.430/0.240
	ctr_prior	0.5582	0.2564	0.2731	0.3331 (0.331–0.335)	0.958	14.25	0.317	0.443/0.267
	recency	0.5141	0.2141	0.2286	0.2913 (0.289–0.293)	0.952	16.30	0.624	0.200/0.346
	bm25	0.5744	0.2741	0.2924	0.3536 (0.351–0.356)	0.896	15.38	0.665	0.354/0.353
	emb	0.6368	0.3063	0.3344	0.3938 (0.391–0.396)	0.829	15.21	0.634	0.405/0.387
	hybrid_rrf	0.6262	0.2978	0.3233	0.3845 (0.382–0.387)	0.853	15.26	0.659	0.391/0.381
	pop_oracle*	0.5976	0.2892	0.3126	0.3722 (0.370–0.374)	0.919	15.24	0.198	0.358/0.381

Semantic retrieval wins on both, by more on MIND (AUC 0.6368 vs BM25 0.5744) than on EB-NeRD (0.5453 vs 0.5377); these are unsupervised similarity scores, not trained rankers, so ~ 0.63 AUC is the honest ceiling of this family. The **beyond-accuracy columns cut the other way**: the embedding ranker has the *lowest* intra-list diversity on both datasets (0.673 / 0.829 against 0.772 / 0.945 for random) — it is accurate because it is narrow. Prior popularity is the opposite failure: on MIND it covers only 35.6% of the catalogue and is worth 0.430 on head impressions against 0.240 on tail ones. Whatever ships needs a diversity constraint the accuracy metrics will never ask for.

5b • Every knob, and what sweeping it was worth

knob	swept over	shipped	best	worth	§
BM25 (k ₁ , b)	4 × 4 grid + BM25L	k ₁ =2.0, b tuned per corpus	MIND b=1.0; EB-NeRD b=1.0 (small) / 0.3 (large)	b: +70% on MIND, 2.3% total spread on EB-NeRD	\$3
ANN index + parameters	flat, HNSW (M, efC, efS), IVF (nlist, nprobe), SQ8, PQ	exact IndexFlatIP	exact below N≈8K; HNSW above	9.4× speed at 125K	\$4
history window	n _{recent} ∈ {5..100, all} × decay	all, no decay (was 30)	all, no decay	eb/emb +5%, eb/bm25 +23%, mi/emb +2%, mi/bm25 +3% vs n=30	\$5b
similarity cutoff	global τ and per-user α-best	none (fixed top-200)	τ tuned per corpus	eb +11%, mi +12% at matched budget	\$5b
stored features	10 user/article features	embeddings only	category _{match} , emb	eb +5.9%, mi +0.4% AUC	\$5b
query-term saturation	k ₃ ∈ {∞, 1000, 32, 8, 2, 0}	∞ (none)	∞ — confirmed	binary costs ebnerd -56%, mind -17%	\$5b
indexed text + tokeniser	title / +abstract / +body × stop × stem	title+abstract, stop+stem	title+abstract — confirmed	ebnerd: stopwords -10%, body -9%; mind: stopwords -26%	\$5b
user representation	1 vector vs k chunks / k-means centroids	1 mean-pooled vector	k-means k=2 — not adopted	ebnerd +3.8% (k=2), mind +2.2% (k=8)	\$5b
history source	shipped vs augmented	shipped	shipped	<0.3% either way	\$6

Cutoff ablation. Sweeping K over {1,3,5,10,20} separates what one nDCG@10 conflates. At K=1 the rankers are furthest apart; by K=20 EB-NeRD's impressions are exhausted (hit@20 = 0.962 even for random, on a median 12-candidate list) and the metric stops discriminating. The datasets then disagree about fusion: on EB-NeRD the rank-fusion hybrid wins every one of them except hit@1 (0.1572 for embeddings against 0.1566) — fusion improves the whole list and still loses the single slot that matters most, while on MIND plain embeddings win every one of them — so a candidate generator (set recall) and a ranker (order) need measuring separately.

6 • Anti-gaming (Q9)

pop_oracle* is pop_prior with one change — clicks counted from *inside* the scored split instead of before it. Same candidates, same metric code: AUC 0.4437→0.6540 (+47%) on EB-NeRD, +12% on MIND — one illegal feature turns a below-chance baseline into the second-best ranker. This is blocked structurally: 10 test functions (146 parametrised cases) in tests/test_no_leakage.py enforce the split boundary — no derived feature computed past its cutoff, no serving-time-unavailable signal (next_read_time, next_scroll_percentage) in the store — with read_time/scroll_percentage stored but excluded from every ranker as a deliberate grey zone.

6b • The same pipeline at Codabench scale

Identical code, identical configs, large variants (ebnerd: 125,541 articles, vocab 82,638, mind: 130,379 articles, vocab 80,855). EB-NeRD large's test split is 12.5M labelled impressions over 150M candidate rows; the ranking harness samples 600,000 impressions because its peak memory is ~74 KB per impression, not because the statistics need help — the 95% CIs are an order of magnitude narrower than the gaps they separate. Retrieval (§3) and both leaderboard submissions run over the full split.

dataset	ranker	AUC small	AUC large	nDCG@10 small	nDCG@10 large
ebnerd	bm25	0.5377	0.5275	0.4617	0.4520
ebnerd	emb	0.5453	0.5462	0.4660	0.4666
ebnerd	pop_prior	0.4437	0.4274	0.3883	0.3804
mind	bm25	0.5744	0.5751	0.3536	0.3535
mind	emb	0.6368	0.6360	0.3938	0.3935
mind	pop_prior	0.5331	0.5450	0.3107	0.3132

On ebnerd the (k₁,b) sweep **reverses** between variants: b=1.0 wins on small and b=0.3 on large (125,541 articles vs 20,738). Length normalisation is a property of the corpus, not of the algorithm, so a parameter tuned on the demo bundle is not transferable to the leaderboard bundle — the one result here that would have cost real leaderboard score if it had been assumed rather than measured.

The rest of the conclusions survive the 6× jump in catalogue size: the ordering of rankers, the below-chance prior-popularity result on EB-NeRD, and the size of the Q9 leak are all reproduced. Three scaling defects had to be fixed to get here, and each is a design lesson rather than a tuning knob — see §7.

6c • Systems ablations: tool, index and storage choices, priced

Each row re-measures a systems claim against this corpus, on the metric the claim is about, and prices retrieval decisions in recall rather than only in milliseconds. Two further results are in the reports: the service-time distribution is M/D/1, not M/M/1 (CV = 0.01, so M/M/1 over-predicts wait 1.9×), and Heaps' law fits either corpus but overshoots +53.6% of the vocabulary when used held-out as a *predictor*. Full write-ups are in reports/12..15.

subsystem	shipped	alternatives measured	engineering metric	verdict
Serving	1 process, 48 leaf threads	1→48 servers	D _{total} 9.19 ms, bottleneck ann _{search}	110→467 q/s (4.23×); serial model within 1.5%
Tail latency	no hedging	hedge + per-leaf budgets	p99 under load	−86% on an idiosyncratic tail, +204% on a shared one

Shard skew	doc-ranged	micro-sharding $n \in \{16, 64, 128\}$	biggest-shard share, p50	worse: share 29.6→18.4% but p50 5.51→8.50 ms
ANN updates	wholesale rebuild	append-in-place, LSM segments	s/week at equal bytes & recall	rebuild 302 s, append 8.8 s , segments 41 s
Storage	vectors in RAM	cold NVMe, random vs sequential	MB/s at fixed volume	cold 1,477 MB/s; random penalty up to 93×
Compression	uncompressed parquet	snappy, lz4, zstd	cold read time	zstd 2.7× smaller, 1.7× <i>slower</i> — decode, not fetch, is the bottleneck
Near-duplicates	none	SHA-256, shingling, MinHash+LSH	share of catalogue, recall@100	$J \geq 0.8$: 2.2% of articles carrying 0.076% of clicks → +0.0003 recall
Seen-set	exact hash set	Bloom, 4-16 bits/key	bytes, recall@100	16 b/key: 0.25× bytes for +0.00002 recall
Query processing	one sparse matmul	TAAT, DAAT, DAAT+WAND	postings touched, multiply-adds	WAND rank-safe, prunes 66% at 859 terms, but does half the arithmetic — BLAS wins at 2,063 docs
↪ at scale	—	universe ×61	WAND work ÷ matmul work	$2.1 \times \rightarrow$ 6.6× : the query path breaks at ~5× the <i>catalogue</i> , not 10× the users
Posting codes	raw uint32	v-byte, bit-packed, Elias-γ, Roaring	bits/gap, decode GB/s	v-byte 12.5 b/gap against the slide's 8-9; break-even needs 2.6 GB/s, numpy gives 0.09
Index build	single-pass in RAM	SPIMI, 20k-doc blocks	peak RSS delta	869.6 MB → 0.8 MB for 1.3× the time
Candidate tier	full universe	5-50% popularity tier	recall@100, docs scored	5% tier: 0.0869→ 0.2799 (+222%) scoring 20× fewer docs
Merge order	—	√L skips; ascending vs descending df	merge steps	skips 10.9×; cheapest-first saves 87%

Four of these change the code: append to the ANN index rather than rebuilding it; build the inverted index with SPIMI; add the 5% popularity tier; Bloom the seen-set once the user count grows. The rest are kept deliberately — the matmul, weight-1 fields, no positions, no compression, no dedup pass — each now with a number behind it rather than a habit. Eight measurement bugs were caught in the harness before they became results, including a MinHash whose k permutations were all the same permutation; a result is only as good as the rig that produced it.

7 • Where this breaks at 10×

- **The join that already broke.** Remapping article ids inside history lists by explode → join → group_by hit 120 GB on EB-NeRD large and died — not from volume but from shape: 200,000 beyond-accuracy rows carry 250 candidates each and all share `impression_id = 0`, so the `group_by` silently merged them. Rewritten as an in-place `list.eval(replace_strict)`, peak RSS is 8.9 GB. The same join bit again: polars returns rows in hash order, and both graders score one line per raw row *in raw order*, which on those rows no sort can recover. The store now carries `src_row`, tested per split.
- **Features freeze at the split boundary.** Popularity is computed once, at the start of a 7-day window, on a corpus whose median clicked article is 80 hours *younger* than that boundary — which is why it scores below random on EB-NeRD. At 10× it does not get worse; it is already maximally wrong. The fix is a rolling in-window update with a strictly causal cutoff.
- **Exact search stops being free.** \$4 puts the crossover at $N \approx 8K$, so the 125K corpus already wants HNSW. The index build becomes the bottleneck: 42 s single-threaded at 125K, superlinear in N .
- **BM25 scoring is a dense matmul in disguise.** $H \cdot TF$ is one product for all users, which is why it is fast; at 10× users × 10× vocabulary the intermediate stops fitting and it must become blocked. \$6c locates the other wall: WAND overtakes it at ~5× the catalogue, so the query path breaks on universe size, not on user count.
- **Single node.** Replication is the one item on L2's list a single box cannot measure, and the wired GPU-engine benchmark (`make bench-gpu`) is unmeasured because RAPIDS would not install over this network — both stated as gaps, not estimated.

8 • Leaderboards

Both submissions build from the raw behaviours files: MIND 2,370,727 and EB-NeRD 13,536,710 lines, one per *raw row* — grouping by `impression_id`, as the starter notebook does, collapses all 200,000 beyond-accuracy rows into one. The first MIND upload was rejected outright for carrying a descriptive zip name; both graders open a fixed inner filename, now asserted by `tests/test_submission_format.py`.

The popularity baseline is degenerate on MIND's test week, and the leaderboard proves it. It scored AUC 0.4900 — below chance — against 0.5440 for the same scorer on dev. Popularity is counted from clicks strictly before the split cutoff, and the test impressions run 2019-11-16-2019-11-22, up to seven days past it: 85.8% of candidate slots score exactly 0 and 31.4% of rows are a complete tie, so a third of the test set is ordered arbitrarily. Ranking the same rows by content instead — cosine between the mean-pooled history vector and each candidate — leaves 1.0% of slots at zero and scored **0.6496** on the same hidden test set, +33% over the popularity file. A stale prior does not degrade gracefully; it stops being a ranking at all.

ID #	File name	Date	Status	Score	Actions
903894	prediction_semantic.zip	2026-08-27 12:23	Finished	0.6496	✓ ↩
903893	prediction.zip	2026-08-27 12:23	Finished	0.4900	📄 ↩ 🔴
903746	prediction.zip	2026-08-27 09:59	Finished	0.4900	📄 ↩ 🔴
902300	prediction.zip	2026-08-26 11:43	Finished	0.4900	📄 ↩ 🔴
902280	mind_popularity_prediction.zip	2026-08-26 11:26	Failed		🔴

MIND leaderboard: three identical popularity uploads at 0.4900, the content submission at 0.6496. The direction of the dev→test move is the argument — popularity *fell* (0.5440→0.4900) while content *rose* (0.6374→0.6496).

EB-NeRD returned no score. Competition 2469's open phase routes to RecSys24 competition v5, the organisers' own queue rather than Codabench's shared pool; an authenticated passive declare returned **41 messages ready to 1 consumer** — ~8 days of backlog at their stated 4-5 h per job. The sanctioned workaround, our own compute worker, is blocked by campus egress rather than by the broker: four external probes reach it in 56-214 ms, while portquiz.net answers here on 8080, not 5672.